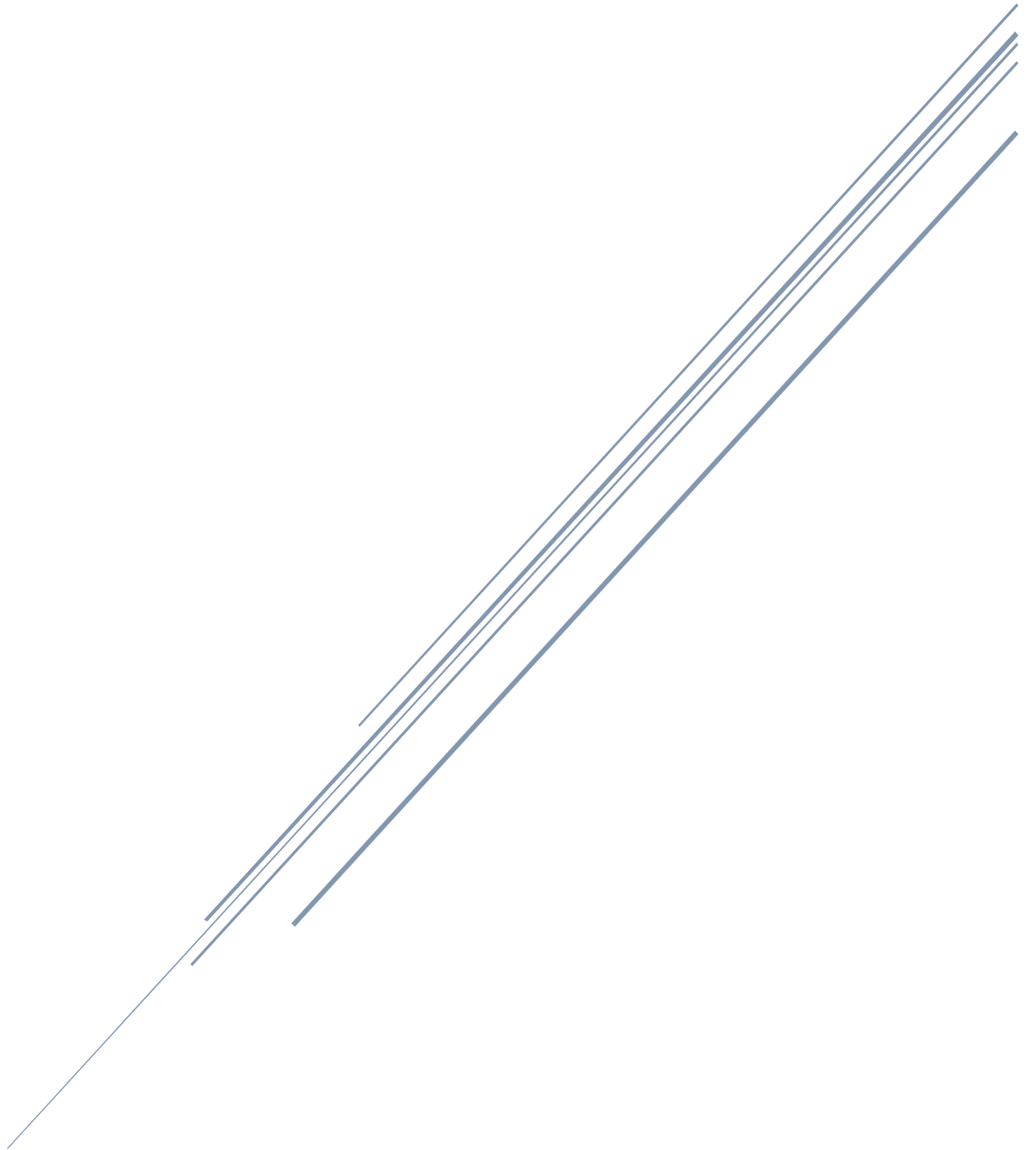


# NESNE TABANLI PROGRAMLAMA

Temel Kavramlar



Vize Hazırlık Ders Notları

## 1. NESNE TABANLI PROGRAMLAMA (OBJECT ORIENTED PROGRAMMING) NEDİR?

Nesne tabanlı programlama, daha öncelerde yaygın olarak kullanılan prosedür tabanlı programlama mantığının yetersiz kaldığı bazı durumları aşmak için geliştirilen bir programlama metodudur.

Prosedür Tabanlı (Yapısal) Programlamanın yetersiz kaldığı durumlar.

- Geliştirilen uygulama parçalanamayan bir bütün halindeydi. Bu yüzden, uygulama üzerinde çalışan her geliştirici; uygulamanın hemen her yapısına hâkim olmalıydı.
- Projelere yeni yazılımcıların katılması önemli bir adaptasyon süreci gerektiriyordu.
- Ufak değişiklikler uygulamanın farklı noktalarında büyük sorunlara yol açabiliyordu.
- Yıllarla birlikte müşteri ihtiyaçları ve donanım kabiliyetleri arttı. Geliştirilen uygulamaların kapsamı ve boyutları büyüdü. Başlanan projelerin çoğu istenen sürelerde yetiştirilememeye ve geliştirme zorluklarından ötürü iptal olmaya başladı.
- Uygulama maliyetleri giderek artmaya başladı.

1960'larda NTP (Object Oriented Programming – Nesne Tabanlı Programlama) fikrini ilk ortaya atan Alan Kay, önerdiği metodolojiyi şu şekilde ifade etmiştir:

- Uygulama, nesneler ve onların ilişkileri çerçevesinde belirli bir iş yapmak için geliştirilebilmelidir.
- Her nesnenin bir sınıfı olmalıdır ve sınıflar nesnelerin ortak davranışlarını ifade etmelidir.
- Nesneler birbirleri ile iletişime geçebilmelidir.

Temel prensibi ise kabaca şu şekilde ifade edilebilir;

- Herşey bir objedir.
- Objeler birbirlerine ulaşan mesajlar yada istekler oluştururlar.
- Her objenin kendi hafızası vardır.
- Her **obje** bir **sınıfın örneğidir**. Sınıf benzer objeleri gruplayan tasarıdır.
- Sınıf, obje ile ilgili davranışların tanımlandığı yerdir.
- Sınıflar tek köklü ağaç yapıları içinde organize edilirler. Bu yapıya **miras hiyerarşisi** denir.

Geleneksel prosedür yönelimli programlama yaklaşımında, bir program gerçekleştirilecek bir dizi işlem adımını, yani bir algoritmayı, tanımlar. Nesneye tabanlı yaklaşımda ise, bir program birbiriyle etkileşim halinde olan bir nesneler sistemini tanımlar.

Nesne Tabanlı Programlamayı, prosedürel programlamada bulunan soyut program geliştirme mantığını rafa kaldırıp, gerçek dünya modellemesi ile program geliştirme çabası olarak da düşünebiliriz.

Gerçek dünya modellemesiyle anlatılmak istenen şudur:

Bir fabrika örneğini ele alalım. Bu fabrikada işçiler, makineler gibi birçok nesne bulunur ve bu nesnelerin ilişkisi çerçevesinde fabrika çeşitli işler yapıp çıktılar üretebilir. NTP ile programlama mantığında da, bu örnekteki benzer şekilde program kurgulanır. Çeşitli nesneler geliştirilip birbirleriyle ilişkilendirilerek, belirli amaçlara hizmet eden uygulamalar geliştirilir.

Bu yapının önemli getirileri şunlardır:

- Yazacağınız sınıflar (class) birbirinden bağımsız olarak geliştirilebilir. Bu sayede program böl, parçala, fethet mantığı çerçevesinde çok kolay bir şekilde parçalanır ve her parça ayrı ayrı ele alınabilir.
- NTP, yapısı gereği kod tekrarlarının önüne geçer (doğru bir şekilde kullanılırsa).
- Bu durum, özellikle ilk dönemlerde yazılımcıların hızlı bir şekilde NTP yapılarına geçmesinin temel nedenlerinden biri olmuştur.
- Projelerin yönetilebilirliğini büyük miktarda artırdığı için daha büyük projeler çok daha az çaba ile yönetilebilir hale gelmiştir. Yine aynı getiriler sayesinde, projeler rahat bir şekilde büyütülebilmektedir.

## Sınıflar (Class)

Nesneye-yönelik programlama, prosedürel soyutlama ve veri soyutlamasını sınıflar biçiminde birleştirir. Bir sınıfı tanımlarken, yüksek-düzeyle soyut bir yapıya ilişkin her şey belirlenir. Bu sınıfa ait bir nesneyi kullanırken, sınıf içinde bildirilmiş veri tipleri ve onlar üzerinde tanımlanmış işlemler göz ardı edilebilir.

NTP temelde nesneler ve onların ilişkisi üzerine kurulu bir metodoloji olarak tanımlanmıştır. Nesneler sınıflardan türetilir ve yetenekleriyle yapabilecekleri sınıflarla belirtilir. Nesne-Sınıf ilişkisi için şöyle bir örnek verilebilir: İnsan bir sınıftır, her bir kişi, insan sınıfının bir nesnesidir. Örneğin, günlük kullanımda Ali bir insandır diyebiliyor olsak da, şu denklem yanlıştır: İnsan=Ali. Eğer sözlükte insan maddesinin yanında tanım olarak Ali'den bahsedilmiyorsa, bu eşitlik hiçbir zaman doğru olamaz. İnsan sınıfı bize Ali gibi her insanın yapabileceklerini, yeteneklerini ve özelliklerini belirtir. İnsan konuşabilir, el sallayabilir, koşabilir v.s. dediğimiz her seferinde, bu işlerin Ali için de geçerli olduğunu kabul etmiş oluruz. Her insanın bir göz rengi vardır dersek, Ali'nin de bir göz renginden bahsedebiliriz.

## Erişim belirleyiciler

Erişim belirleyiciler (access modifiers), sınıflara nerelerden ve ne şekilde erişileceğini belirtir.

- Public: Her yerden erişilebilir.
- Private: Sadece tanımlandığı sınıf içerisinden erişilebilir.
- Internal: Sadece bulunduğu projede erişilebilir.
- Protected: Sadece tanımlandığı sınıfta ya da o sınıfı miras alan sınıflardan erişilebilir.

- Protected Internal: Sadece tanımlandığı sınıfta ya da o sınıfı miras alan sınıflardan erişilebilir. Ayrıca tanımlamanın aynı proje içerisinde olma şartı yoktur. Protected'dan farkı budur.

Eğer erişim belirleyici belirtilmemişse, sınıflar internal'dir.

## 2. NESNE TABANLI PROGRAMLAMANIN TEMEL İLKELERİ

Nesneye-yönelik programlamanın 4 temel ilkesi bulunmaktadır.

- Soyutlama (Abstraction)
- Sarmalama (Encapsulation)
- Kalıtım (Inheritance)
- Çok biçimlilik (Polymorphism)

### 2.1. SOYUTLAMA (Abstraction)

“Soyutlama” , belirli bir bakış açısından, önemli özelliklere odaklanabilmek için ayrıntıları göz ardı etme sürecidir.

Geleneksel olarak, bir programlama dili soyutlama yapmaya izin verdiği ölçüde yüksek-düzeyle (high-level) kabul edilir.

C# (ve diğer nesneye-yönelik programla dilleri) verilen bir işi C'den daha soyut bir tarzda tanımlama imkanı verir.

### 2.2. SARMALAMA/KAPSÜLLEME (Encapsulation)

Programımızın tasarımını kendi işlem kümelerine sahip soyut veri tipleri etrafında yaparak kendimizi kodlama / gerçekleştirme detaylarından daha fazla arındırırız. Bu da bizi nesneye-yönelik programlamanın bir diğer avantajına, sarmalamaya, götürür.

“Sarmalama”, soyutlamayı desteklemek ya da güçlendirmek için bir sınıfın iç yapısının gizlenmesidir. Bu gizleme, bir sınıfın “görünür” arayüzü ile “özel” gerçekleştirmesi arasında keskin bir ayrım yapmamızı gerektirir. Bir sınıfın arayüzü, o sınıfın ne yapabileceğini, gerçekleştirmesi ise bunu nasıl yapabileceğini gösterir. Gerçek bir sarmalama, verileri fonksiyonlarla gizlemeyi gerektirir.

Bir insanı örnek verelim. Vücudumuzun yaptığı işler sarmalanmıştır ve biz buna müdahale edemeyiz. Örneğin, biz sadece solumakla ilgileniriz; ciğeri şişirmek, kana oksijen karıştırmak, kirli kanı temizlemek gibi işlemler içeride yapılıyor olsa da, bizim için durum sadece solumaktan ibarettir. Diğer işlemler sarmalanmıştır/kapsüllenmiştir.

### 2.3. KALITIM / MİRAS ALMA (Inheritance)

Kalıtım nesne yönelimli programlamada önemli bir özelliktir. Kalıtım yolu ile eldeki sınıflardan yeni sınıflar türetilir. Türeyen sınıflar türedikleri sınıfın özelliklerini

kalıtım yoluyla devralırlar ve kendisi de yeni özellikler tanımlayabilir. Türetme ile sınıflar arasında hiyerarşik bir yapı kurulabilir.

Bir sınıf hiyerarşisi tanımlamanın 2 pratik faydası vardır.

- Türetilmiş sınıf üst-sınıfın kodunu paylaşabilir;
- Türetilmiş sınıf üst-sınıfın arayüzünü paylaşabilir.

İnsan – memeli ilişkisinde, insanın memeli sınıfını miras aldığı söylenebilir. Bu sayede insan sınıfını yazarken memelilerin özelliklerini tekrar yazmamıza gerek kalmaz. Elinizde bir taşıt sınıfı varsa; otomobil, kamyon, motosiklet gibi alt sınıfları üretmek çok daha az çaba gerektirir.

## 2.4. ÇOK BİÇİMLİLİK (Polymorphism)

Genel anlamı ile bir adın birbiriyle alakalı fakat teknik açıdan farklı iki veya daha fazla amaç için kullanılabilmesi yeteneğidir.

NTP’de ise oluşturulan nesnelerin gerektiğinde başka bir nesne gibi davranabilmesine denir. Çok biçimlilikle programdaki her nesne kendi davranışını değiştirmeden, kalıtım hiyerarşisine göre farklı biçimlerde görülebilir.

Kısaca Nesne Tabanlı Programlamayı özetleyecek olursak;

- NTP programlama dillerine eklenecek kadar kolay bir özellik değildir. Yeni bir düşünme yoludur!
- NTP, programa birbiri ile ilişkideki topluluklar gözü ile bakar. Belli bir amaç için var olan ve bir görevi olan her şeye obje (nesne) der.
- Obje veri ve davranışları ile kapsüllenmiş (insan vücudu örneği) olarak oluşur.
- **Sınıf**, bir grup nesne tarafından paylaşılacak olan yapı ve davranışları tanımlar.
- Belli bir sınıfın her nesnesi, sanki o sınıfın bir kalıp ile damgalanmış gibi, sınıf tarafından tanımlanan yapı ve davranışı gösterir → **Sınıf örnekleri, İnsan Sınıfı**
- Sınıfın kod ve verilerine **üye** denir.
- Sınıfın davranışları metotlar tarafından tanımlanır.
- **Miras**, bir nesnenin diğer bir nesnenin özelliklerini kazanmasıdır.
- **Polimorfizm** ise *bir arabirim- çok metot* kavramı ile açıklanır. Aynı metodun farklı alıcılara göre farklı davranması
  - Köpek – koklama
    - Koklama(kedi) → havlar, kovalar
    - Koklama (yiyecek) → salya akar, kaba koşar.

### 3. DEĞİŞKENLER VE VERİ TİPLERİ

Değişken, programın çalışması için gerekli verilerin tanımlanarak, bellek üzerinde tutulduğu bölgelere verilen isimlerdir. C#'da değişkenler kullanılmadan önce tanımlanırlar. Tanımlama o bellek bölgesinde tutulacak olan **verinin türünün** belirtilmesidir. Bir değişken tanımlandıktan sonra **aynı türden** değer atamak koşuluyla değeri değiştirilebilir. C#'da değişken tanımlama şu şekilde yapılır;

- veritipi değişkenadı;

Örnek verecek olursak,

- `int i;`
- `char karakter;`
- `string deger;`

gibi tanımlamalar yapılabilir. Değişken tanımlarken, aynı zamanda aynı türden birden fazla değişkeni aynı satırda tanımlayabiliriz. Bunun yanısıra değişken tanımlarken tanımladığımız değişkene değer atayabiliriz.

- `int sayi1, sayi2, sonuc;`
- `int sayi1=28;`
- `int sayi1, carpan=3, sonuc;`

#### 3.1. Değişken İsimlendirme Kuralları

Değişkenler isimlendirilirken kelimeler arasında boşluk karakteri kullanılmaz. Gerekli durumlarda `_` (alt tire) ile kelimeler birleştirilebilir.

C#'da değişken isimleri büyük-küçük harf duyarlıdır. Yani `sayi` ile `SAYI` aynı değişkeni göstermez.

Değişken isimlerinde **zorunluluk olmamasına karşın** Türkçe karakter kullanılmamalıdır.

Değişken isimleri `?, !, :, %` gibi özel karakterler içeremez.

Değişken ismi olarak C# dilindeki özel kelimeler seçilemez.

Değişken isimlerinde **zorunlu olmamasına karşın** küçük harf kullanımı tercih edilir. değişken ismi iki ya da daha fazla kelimedenden oluşuyorsa ilk kelime hariç diğer kelimelerinin ilk harfinin büyük yazılması geleneksel bir kuraldır. (`sayi`, `maasMiktari`, `kitapSayisi` v.b.).

#### 3.2. Değişken Tipleri

Değişkeni kullanmak istediğimiz duruma göre farklı değişken tipleri vardır.

##### 3.2.1. Sayısal veri tipleri

Üzerinde işlem yapabileceğimiz sayısal veri tipleridir. Örneğin;

- `int a1=5, a2=7;`

şeklinde tanımladığımız iki değişkeni toplarsak,  $a1+a2$  işleminin sonucu “12” olur.

| Tür     | Boyut   | Kapasite                                                              | Örnek                                           |
|---------|---------|-----------------------------------------------------------------------|-------------------------------------------------|
| byte    | 1 bayt  | 0, ..., 255 (tam sayı)                                                | byte a=5;                                       |
| sbyte   | 1 bayt  | -128, ..., 127 (tam sayı)                                             | sbyte a=5;                                      |
| short   | 2 bayt  | -32768, ..., 32767 (tam sayı)                                         | short a=5;                                      |
| ushort  | 2 bayt  | 0, ..., 65535 (tam sayı)                                              | ushort a=5;                                     |
| int     | 4 bayt  | -2147483648, ..., 2147483647 (tam sayı)                               | int a=5;                                        |
| uint    | 4 bayt  | 0, ..., 4294967295 (tam sayı)                                         | uint a=5;                                       |
| long    | 8 bayt  | -9223372036854775808, ..., 9223372036854775807 (tam sayı)             | long a=5;                                       |
| ulong   | 8 bayt  | 0, ..., 18446744073709551615 (tam sayı)                               | ulong a=5;                                      |
| float   | 4 bayt  | $\pm 1.5 \cdot 10^{-45}$ , ..., $\pm 3.4 \cdot 10^{38}$ (reel sayı)   | float a=5F; veya float a=5f;                    |
| double  | 8 bayt  | $\pm 5.0 \cdot 10^{-324}$ , ..., $\pm 1.7 \cdot 10^{308}$ (reel sayı) | double a=5; veya double a=5d; veya double a=5D; |
| decimal | 16 bayt | $\pm 1.5 \cdot 10^{-28}$ , ..., $\pm 7.9 \cdot 10^{28}$ (reel sayı)   | decimal a=5M; veya decimal a=5m;                |

### 3.2.2. Metinsel veri tipleri

Metin tipindeki verileri içerir. Örneğin 5 sayısını a1 değişkenine string olarak atayalım ve a2 değişkenine de 7 sayısını string olarak atayalım.

- `string a1="5", a2="7";`

Bu iki değişkenimiz üzerinde  $a1+a2$  işlemini yaparsak, yaptığımız işlem sayısal değil string bir işlem olduğundan elde edeceğimiz sonuç iki değişkenin yan yana gelmesiyle “57” olur.

| Tür    | Boyut    | Açıklama                 | Örnek                                    |
|--------|----------|--------------------------|------------------------------------------|
| char   | 2 bayt   | Tek bir karakteri tutar. | char a='h';                              |
| string | Sınırsız | Metin tutar.             | string a="Buraya Bir Metin Gelecektir."; |

### 3.2.3. Boolean

Koşullu yapılarda kullanılır. Boolean türünden değerlere true, false gibi ifadeler örnek verilebilir. Tanımlaması değişken tanımlamasıyla aynıdır. Bazen true/false yerine mantıksal bir deyim de atanabilir.

- `bool mezun;`
- `bool mezun=true;`
- `bool emekli=false;`
- `bool bformul=(x>15&& x<30);`

### 3.2.4. Object

Bu değişken türüne her türden veri atanabilir.

### 3.2.5. Sabitler

Eğer program boyunca bir veri tipinde sabit bir değer olacak ise bu değer değişken olarak değil, sabit veri tipi olarak tanımlanır. En klasik örnek olarak pi sayısı verilebilir. Pi sayısı kesirli sayı olduğundan int değil double ile tanımlanır. Sabit verilerimize program bloğunda değer atanamayacağından, **tanımlama yapılırken değer atanması** gerekir.

- `const double pi=3.14;`

## 3.3. Operatörler

Operatörler işlevlerine göre 6 kısımda incelenebilir. Bunlar;

- Aritmetik operatörler.
- Karşılaştırma operatörleri.
- Bitsel operatörler.
- Mantıksal operatörler.
- Atama ve işlemleri atama operatörleri.
- Özel amaçlı operatörler.

### 3.3.1. Aritmetik Operatörler

`+`, `-`, `*`, `/`, `++`, `--` ve `%` aritmetik operatörlerdir.

`/`, `*`, `+` ve `-` operatörleri matematiksel işlemler yapar.

`%` operatörü ise bölümden sonra kalanı bulmak yani "mod" almak için kullanılır.

`++` ve `--` işlemin uygulandığı sayıyı 1 artırır veya 1 azaltır.

`*`, `/`, `%` operatörleri aynı satırdaysa eşit önceliğe sahiptir. `+` ve `-` operatörleri de aynı satırdaysa eşit önceliklidir fakat `*`, `/`, `%` operatörlerinin `+` ve `-` operatörlerine göre önceliği vardır.

### 3.3.2. Karşılaştırma Operatörleri

C# ta 6 adet karşılaştırma operatörü vardır. Bunlar; `<`(küçüktür), `>`(büyüktür), `<=`(küçük veya eşittir), `>=`(büyük veya eşittir), `==`(eşittir), `!=`(eşit değildir).

Karşılaştırma operatörleri aritmetik operatörlerden düşük önceliklidir.

`<`, `>`, `<=`, `>=` operatörleri `==` ve `!=` operatörlerine göre önceliklidir fakat aynı satırda olduklarında eşit önceliğe sahiptir.



### 3.3.3. Bitsel Operatörler

Bitsel operatörler  $\&$ (bitsel ve),  $\sim$ (bitsel değil),  $|$ (bitsel veya),  $\wedge$ (bitsel özel veya) operatörleridir. Sayıların kendileri yerine bitlerini kullanan operatörlerdir.

$\&$ (ve) işlemi karşılıklı basamaklardan her ikisi de 1 ise ilgili basamağında 1 tutar, diğer durumlarda 0 tutar.

$\sim$ (değil) operatörü operandının her basamağını tersleştirip tutar.

$|$ (veya) işlemi karşılıklı basamaklarından herhangi birisi 1 ise ilgili basamağında 1 tutar, diğer durumlarda 0 tutar.

$\wedge$ (özel veya) işlemi karşılıklı basamakları farklıysa ilgili basamağında 1 tutar, diğer durumlarda 0 tutar.

Bitsel operatörler aritmetik operatörler ve karşılaştırma operatörlerinden düşük önceliklidir fakat bu operatörler aynı satırda olduklarında aralarındaki öncelik sıralaması  $\&$ (bitsel ve),  $\wedge$  (bitsel özel veya),  $|$ (bitsel veya) şeklindedir.

### 3.3.4. Mantıksal Operatörler

C#’ ta 3 adet mantıksal operatör bulunmaktadır. Bunlar;  $\&\&$ (ve),  $!$ (değil),  $||$ (veya) operatörleridir.

$||$ (veya) operatörü iki değerden birisi doğru ise doğru, ikisi de yanlış ise yanlış değerini döndürür ayrıca önceliği en az olan mantıksal operatördür.

$\&\&$ (ve) operatörü iki değerden ikisi de doğru ise doğru, en az birisi yanlış ise yanlış değerini döndürür.

$\&\&$ (ve),  $||$ (veya) operatörleri aritmetik, karşılaştırma ve bitsel operatörlere göre düşük önceliklidir fakat kendi aralarındaki öncelik sıralaması  $\&\&$ (ve),  $||$ (veya) şeklindedir.

$!$ (değil) operatörü aritmetik operatörlerden  $(++)$ arttırma ve  $(--)$ eksiltme operatörleri ile eşit önceliklidir.

### 3.3.5. Atama ve İşlemlili Atama Operatörleri

$=$ (atama) operatörü bir değişkene herhangi bir değer atamak için kullanılır.

$*=$ ,  $/=$ ,  $+=$ ,  $-=$ ,  $\&=$ ,  $\wedge=$ ,  $|=$  gibi operatörlere işlemli atama operatörü denir. Çok yaygın kullanımı olmamakla birlikte kullanımı şu şekildedir;

$a=a+b$  yerine  $a+=b$

$a=a/b$  yerine  $a/=b$

$a=a^b$  yerine  $a^b=b$

Atama ve işlemli atama operatörleri aynı satırda yer alıyorsa öncelik sıralamaları soldan sağa eşittir fakat genel olarak bakıldığında bu operatörler öncelik sıralaması bakımından en son sırada yer alır.

### 3.3.6. Özel Amaçlı Operatörler

- **?: operatörü:** C#' ta üç operand alan tek operatördür. Kullanımı şu şekildedir;  
koşul ? doğru\_değer : yanlış değer
- **( ) tür dönüştürme operatörü:** Tür değiştirmek için kullanılan bir operatördür.  
(değiştirilecek tür) değişken\_yada\_sabit
- **[ ] operatörü:** Dizilerde elemanın dizinini belirtmek için kullanılır.
- **+ ve - operatörü:** Bir değişkenin pozitif veya negatif olmasını sağlar. Ayrıca + operatörü iki string(katar)'ı birleştirmekte de kullanılır.
- **typeof operatörü:** Herhangi bir değişken türünün CTS (Common Type System - Ortak Tip Sistemi) karşılığını type(tip) olarak tutar.
- **sizeof operatörü:** Temel türlerin ve yapıların bellekte ne kadar alan kapladıklarını verir. "sizeof" operatörü sınıflar için kullanılamaz ancak tanımlanacak yapılar için kullanılabilir.
- **new operatörü:** Yeni bir nesne oluşturmak için kullanılan operatörlerdir.

## 4. C#'DA KARAR YAPILARI VE DÖNGÜLER

Programın akışında bir sonucun başka bir duruma bağlı olduğu durumlarda veya başka bir şarta bağlı olarak yapılacak işlemlerde biz karar yapıları ve döngüleri kullanırız. Bu döngü ve karar yapılarında değişkenler üzerinde işlem yapmadan önce kullanacağımız kıyaslamalarda **Karşılaştırma Operatörlerini** (<, >, <=, >=, ==, !=) kullanırız.

Karar yapıları ve döngülerin kullanımı ile ilgili derslerimizde uygulamalar yapacağımızdan sadece tanımlamaları yapılmış, bu ders notunun kapsamının genişletilmemesi için örneklere yer verilmemiştir.

### 4.1. if-else Deyimi

Program içinde karar vermek için kullanılan en basit yapıdır. If..Else deyimi bir Boolean ifadeyi değerlendirerek program denetimi belirtilen deyim gruplarına geçirir.

### 4.2. switch...case Deyimi

Switch deyimi de if deyimi gibi programın koşullara göre yönlenmesini sağlar. Özellikle bir değişkenin değerine göre değişik işlemler yapılacaksa o zaman Switch..Case yapısı uygun olur.

### 4.3. while Döngüsü

Belirtilen koşul gerçekleşene kadar bir döngüye ihtiyacımız olduğunda kullanacağımız döngülerdir. Tek dikkat etmemiz gereken tanımlanan koşul gerçekleşmişse döngünün execute edilmesidir. Bunun nedeni koşulun döngü başlangıcında bildirilmesidir.

#### 4.4. do...while Döngüsü

While döngüsünden tek fark olarak karşılaştırmanın döngünün sonunda yapılmasıdır.

#### 4.5. For Döngüsü

Bir grup deyimi belli sayıda çalıştırarak (tekrar ederek) bir döngü oluşturur. Örneğin birden ona kadar (on kere) şu işlemi yap şeklinde. Özellikle bir döngü denetim değişkeninin kullanıldığı (sayaç) bu döngü yapısı iç içe döngülerin de yapılmasını sağlar.

#### 4.6. Atlama (Jump) Deyimleri

Döngüler ve karar deyimleri görevlerini yerine getirirken, bazen çıkış yapmaya gereksinim duyulur. C#'da goto, break ve continue deyimleri atlama deyimleri olarak bilinir.

- **goto:** Program içinde belirtilen bir etikete atlamayı sağlar.
- **break:** En yakın program bloğuna çıkışı sağlar.
- **continue:** Yeni bir döngüye baslar